

PATH PLANNING ALGORITHMS

INF 421 PI

15 février 2026

Ewan MAGDELAINE
Loïc FERON



TABLE DES MATIÈRES

1	Introduction	2
2	L'algorithme "Particle Swarm Optimization".	3
2.1	Présentation du principe général.	3
2.2	Implémentation pour notre problème.	3
2.3	Analyse de l'algorithme.	5
2.3.1	Choix des hyper-paramètres.	6
2.4	Améliorations et heuristiques.	6
2.4.1	Random restart	6
2.4.2	Simulated annealing	7
2.4.3	dimensional learning	8
2.4.4	Nos propres heuristiques	9
3	L'algorithme "Rapidly-Exploring Random Tree".	10
3.1	Présentation du principe général.	10
3.2	Implémentation pour notre problème.	10
3.3	Analyse de l'algorithme.	13
3.3.1	Résultats.	13
3.4	Améliorations et heuristiques.	14
3.4.1	Path optimization	14
3.4.2	Analyse des performances et Comparaison avec le RRT basique.	15
3.4.3	Intelligent sampling	15
3.4.4	Comparaison avec PSO et Conclusion sur l'algorithme	16
4	Vers une adaptation aux situations réelles : la gestion de plusieurs robots.	17

1

INTRODUCTION

Avec le développement de la robotique et l'automatisation des usines, il est nécessaire de concevoir des algorithmes performants permettant aux robots de trouver leur chemin, le plus court possible, tout en évitant des obstacles. Nous nous sommes donc intéressés à deux algorithmes différents afin de résoudre ce problème. Nous les avons analysés, implémentés en Java et améliorés avec des heuristiques. Nous présentons ici ces analyses ainsi que les résultats.

Nous avons choisi d'implémenter ce projet en Java. Pour la visualisation, nous avons utilisé la librairie Javafx. Nous avons donc créé une classe **Environnement** qui contiendra la liste des robots, des obstacles, ainsi que les dimensions de l'environnement. L'extraction des données d'un fichier texte initialise alors une instance de cette classe **Environnement**.

Nous avons travaillé en collaboration sur github et nous avons utilisé l'IA pour l'importation des librairies et les problèmes liés à l'architecture de notre projet. Par exemple, nous avons commencé par télécharger les fichiers nécessaires pour utiliser javafx sur notre ordinateur mais afin de pouvoir partager le projet, nous avons décidé de migrer sur maven à l'aide de l'IA.

2

L'ALGORITHME "PARTICLE SWARM OPTIMIZATION".

2.1 PRÉSENTATION DU PRINCIPE GÉNÉRAL.

Notre premier algorithme se nomme PSO et va fonctionner de manière itérative. En voici une brève ébauche. On va noter k l'étape d'itération. Soit S le nombre de particules, qui est fixé et inchangé tout au long de l'algorithme. Les positions des particules sont évaluées à l'aide d'une fonction de fitness, que l'on cherche à minimiser, et qui nous fournira le chemin le plus court entre deux points définis préalablement par les fichiers de Scenario.

À l'itération k , une particule i est caractérisée par sa position x_i^k et sa vitesse v_i^k . Chaque particule mémorise la meilleure position qu'elle a rencontrée jusqu'à présent, appelée meilleur local et notée p_i^k . De plus, l'ensemble des particules conserve la meilleure position globale observée, notée g^k .

Les positions et les vitesses sont mises à jour selon les équations suivantes :

$$v_i^{k+1} = w v_i^k + c_1 r_1 (p_i^k - x_i^k) + c_2 r_2 (g^k - x_i^k) \quad (1)$$

$$x_i^{k+1} = x_i^k + v_i^{k+1} \quad (2)$$

où c_1 et c_2 sont des coefficients d'accélération contrôlant respectivement l'influence du meilleur local et du meilleur global. Les variables r_1 et r_2 sont des variables aléatoires tirées uniformément dans l'intervalle $[0, 1]$. Le paramètre w représente le poids d'inertie appliqué à la vitesse.

Les positions et les vitesses peuvent être contraintes à appartenir à des intervalles bornés. Les paramètres S , c_1 , c_2 et w constituent les hyperparamètres de l'algorithme.

Après un nombre fixé d'itérations, qui sera ici S^2 , l'algorithme retourne la meilleure position globale obtenue.

2.2 IMPLÉMENTATION POUR NOTRE PROBLÈME.

Dans notre problème, les particules seront les chemins possibles, croisant un obstacle ou non. Ces chemins sont composés d'un nombre fixe de segment *intermédiaire steps*. La fonction fitness sera la longueur du chemin augmentée d'une pénalité de 20 000 pour chaque obstacle traversé par ce chemin (sachant que $X_{\max}=Y_{\max}=1000$, une telle pénalité est extrêmement handicapante), ce qui nous assure que si un chemin ne croisant pas d'obstacle est trouvé, alors il sera immédiatement choisi.

Pour pouvoir représenter nos particules, nous allons utiliser des arrays listes, chaque particule aura donc une array liste pour la position, et une pour la vitesse.

Pour pouvoir implémenter notre algorithme PSO, nous avons besoin d'une méthode pour vérifier qu'un chemin n'entre pas en collision avec un obstacle. On a toutes les positions d'un chemin i choisi qui sont stockés dans une liste. On va donc raisonner segment par segment (donc en itérant sur k et en prenant les deux extrémités du segment x_i^k et x_i^{k+1}). Pour ce segment donc, nous allons considérer chaque obstacle présent dans le domaine visible. à ce moment là, on considère les bordures de l'obstacle et on vérifie (avec des boucles si alors qui sont assez nombreuses car il y a beaucoup de cas à traiter, même si le fait que les obstacles soient rectangulaires simplifie le problème) si le segment considéré traverse une des bordures de l'obstacle, si c'est le cas on incrémente un indice $nbobstacle[i]$ dans une liste donnant le nombre d'obstacles traversé par chaque chemin. Enfin, on va vérifier que le segment ne se situe pas dans l'obstacle considéré (on vérifie que les deux extrémités ne sont pas dans le cadre formé par les bordures). On a alors la somme des nombres d'obstacles croisés par chaque segment.

Si on souhaite juste savoir si le chemin croise ou non un obstacle, il suffit juste de vérifier que $\text{nbobstacles}[i]$ est non-nul, ce qui signifierait que le chemin i croise bien un obstacle.

On se propose maintenant de donner le pseudo-code de notre algorithme PSO :

Entrée :

- une fonction de fitness f à minimiser
- un nombre de particules S
- un nombre maximal d'itérations $\text{max_iter} \leftarrow S^2$
- des coefficients w, c_1, c_2
- un domaine de recherche borné et deux couples (x, y_e) et (x_s, y_s) qui donnent les points de départ et d'arrivée.

Sortie :

- le meilleur chemin global g^K sous la forme d'une liste décrivant la particule optimale

Algorithme :

Pour chaque particule $i \in \{0, \dots, S-1\}$, faire :

x_i^0 dans le domaine de recherche de manière aléatoire pour k allant de 0 à *intermédiaire steps*

v_i^0 qui est initialisé en 0 pour k allant de 0 à *intermédiaire steps*

$p_i^0 \leftarrow x_i^0$

$g^0 \leftarrow \arg \min_i f(p_i^0)$

$k \leftarrow 0$

Tant que $k < \text{max_iter}$, faire :

Pour i allant de 0 à $S-1$ faire :

Générer r_1, r_2 tirés uniformément dans $[0, 1]$

$v_i^{k+1} \leftarrow w v_i^k + c_1 r_1 (p_i^k - x_i^k) + c_2 r_2 (g^k - x_i^k)$

$x_i^{k+1} \leftarrow x_i^k + v_i^{k+1}$

Contraindre x_i^{k+1} et v_i^{k+1} au cadre autorisé initialement

Si $f(x_i^{k+1}) < f(p_i^k)$ *alors* :

$p_i^{k+1} \leftarrow x_i^{k+1}$

Sinon :

$p_i^{k+1} \leftarrow p_i^k$

FinSi

FinPour

$g^{k+1} \leftarrow \arg \min_i f(p_i^{k+1})$

$k \leftarrow k + 1$

FinTantQue

Renvoyer g^K , le meilleur chemin trouvé par itération

2.3 ANALYSE DE L'ALGORITHME.

Intéressons-nous à présent à la complexité de cet algorithme. Notons m le nombre d'obstacles représenté dans notre problème. Nous avons considéré que notre nombre maximum d'itérations est en S^2 , ce qui est une décision arbitraire mais qui semble fonctionner convenablement. Dans le pire des cas, la complexité de la fonction qui donne le nombre de collisions entre un chemin existant et les obstacles est en $\mathcal{O}(mS^2)$. Or, on a S^2 passages dans la boucle while, et pour chacun de ces passages, on a un test de présence d'obstacles dans la fonction fitness, comme on met à jour toutes les particules à chaque fois on a une complexité en $\mathcal{O}(mS^3)$ à chaque passage dans la boucle.

Donc, la complexité totale est en $\mathcal{O}(mS^5)$ pour notre algorithme, soit une complexité polynomiale en S et en m .

Nous présentons ici pour chacun des cinq scenarios le chemin trouvé par notre algorithme, ainsi que le temps du CPU nécessaire.

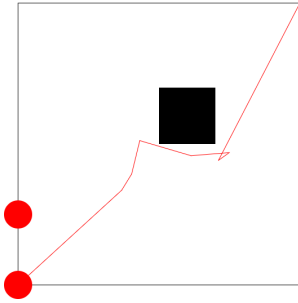


FIGURE 1 – Chemin de longueur 1688.6 calculé par l'algorithme PSO pour le Scenario 0, le temps de calcul était de 102 ms.

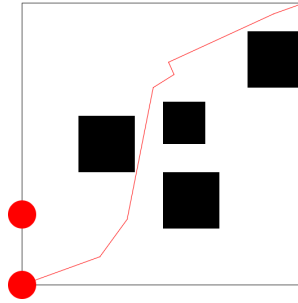


FIGURE 2 – Chemin de longueur 1594 calculé par l'algorithme PSO pour le Scenario 1, le temps de calcul était de 169 ms.

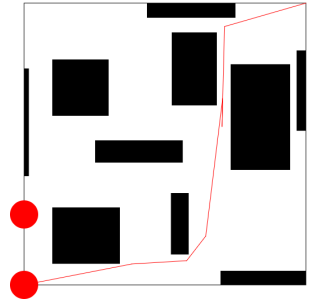


FIGURE 3 – Chemin de longueur 1941.9 calculé par l'algorithme PSO pour le Scenario 2, le temps de calcul était de 152 ms.

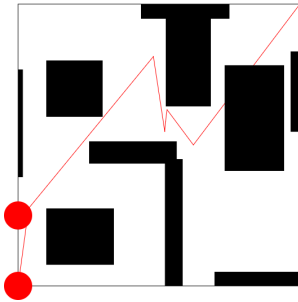


FIGURE 4 – Chemin de longueur 22105 calculé par l'algorithme PSO pour le Scenario 3, le temps de calcul était de 194 ms.

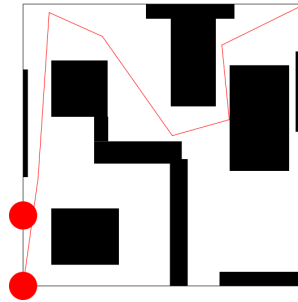


FIGURE 5 – Chemin de longueur 2417 calculé par l'algorithme PSO pour le Scenario 4, le temps de calcul était de 252 ms.

Bien entendu, ces résultats individuels varient mais nous avons présenté ici des résultats représentatifs de ce que l'on obtenait. Notons que la longueur du chemin du scenario 3 prend en compte cette pénalité de 20000 pour chaque obstacle rencontré.

2.3.1 • CHOIX DES HYPER-PARAMÈTRES.

Nous avons fait de nombreux tests pour évaluer quelles valeurs d'hyper-paramètres seraient idéales. Voici les valeurs finales :

$$S = 200 \quad w = 0.9 \quad c1 = 1.8 \quad c2 = 1.0 \quad \textit{intermédiaire_steps}=5$$

Pour expliquer le choix de ces valeurs, nous devons présenter les principaux problèmes rencontrés lors de nos tests. Le principal problème était le blocage dans un minimum local car nos particules n'exploraient pas assez l'espace des chemins. Elles étaient beaucoup trop attirées vers le meilleur chemin qui au début a de grandes chances de contenir un ou plusieurs obstacles. Ainsi, notre but principal est de favoriser une grande exploration pour nos particules.

Analysons maintenant l'influence de chaque paramètre :

- w : ce paramètre gère la conservation de la vitesse. Il ne peut être supérieur à 1 sinon la vitesse va croître exponentiellement et donc plaquer nos particules contre les bords.
- $c1$: ce paramètre gère l'attraction d'une particule vers sa meilleure position. Il est important qu'une particule n'explore pas aveuglément mais puisse revenir à un endroit "correct".
- $c2$: ce paramètre gère l'attraction d'une particule vers la meilleure position globale. Il est important de limiter cet impact afin de ne pas converger vers un minimum local de la fonction objectif.
- S : l'influence de ce paramètre est plus délicate. Le choix réside dans le compromis suivant. Si l'on se fixe un temps d'exécution donné, on peut augmenter S mais alors nous effectuerons moins d'étapes pour chaque particule, ou réduire S et effectuer beaucoup d'étapes par particule. On peut estimer que réduire S permettra d'avoir une solution qui ne rencontre pas d'obstacles plus optimisée mais donne moins de chance de trouver une solution. Au contraire, augmenter S doit permettre de trouver une solution sans obstacle mais devrait être moins optimisée.
- *intermediaire steps* : ce paramètre est en pratique le plus critique car une valeur trop élevée rend l'espace d'exploration trop grand pour obtenir des résultats satisfaisants. Nos scénarios étant assez simple nous avons fixé ce paramètre à 5.

Le choix a été fait de maintenir une vitesse élevée pour explorer le plus de chemin, le scénario 4 ayant plusieurs couloirs très fins, il est difficile de trouver une solution sans obstacles. Pour cette même raison, le choix de S a été augmenté de 50 à 100 puis finalement à 200. Enfin, nous avons initialement testé notre algorithme avec des valeurs de $c1$ et $c2$ égales mais nous avons opté pour un rapport proche de 2 afin de favoriser l'exploration individuelle encore une fois afin de trouver un chemin dans le scénario 4.

2.4 AMÉLIORATIONS ET HEURISTIQUES.

2.4.1 • RANDOM RESTART

On va maintenant tenter d'améliorer notre algorithme précédent en implémentant la modification suivante : lorsqu'aucune amélioration du meilleur global n'est observée pendant plusieurs itérations, certaines particules sont réinitialisées aléatoirement afin d'explorer de nouvelles zones de l'espace de recherche. L'objectif est ainsi d'éviter un phénomène de convergence vers un minimum local. Cela permet à nos particules d'explorer plus d'espace et augmente donc les chances de trouver un chemin sans obstacles.

On propose ainsi de rajouter ceci à chaque itération de la boucle while dans notre algorithme :

```

Si le meilleur global n'a pas été amélioré, alors :
    incrémenter le compteur de stagnation
Sinon :
    remettre le compteur de stagnation à zéro
FinSi
Si le compteur de stagnation dépasse un seuil fixé, alors :
```

réinitialiser aléatoirement une partie des particules et stocker les indices des particules réinitialisées
 remettre leurs vitesses à zéro
 réinitialiser leurs meilleures positions locales
 remettre le compteur de stagnation à zéro
 FinSi

Avec cette amélioration, les résultats que nous obtenons sont les suivants :

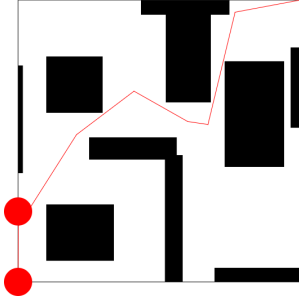


FIGURE 6 – Chemin de longueur 1773.7 calculé par l'algorithme PSO pour le Scenario 3, le temps de calcul était de 3250 ms.

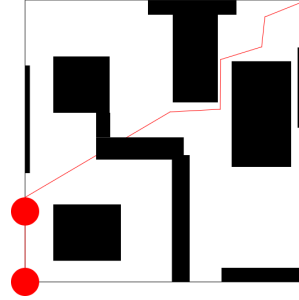


FIGURE 7 – Chemin de longueur 21672 calculé par l'algorithme PSO pour le Scenario 4, le temps de calcul était de 2950 ms.

En comparant avec les performances obtenues sans Random Restart, on constate que cet algorithme est un peu plus lent (on va à chaque fois au bout du compteur max iter) mais le chemin trouvé est effectivement plus court, ce qui témoigne de l'utilité de l'amélioration. Essayons maintenant de faire encore mieux.

2.4.2 • SIMULATED ANNEALING

Néanmoins, un autre problème se pose, on pourrait se retrouver bloqué dans un optimum local, et ne jamais en sortir, ce qui nuit à la performance globale de l'algorithme. Afin d'éviter ceci, on va rajouter la modification suivante : une solution candidate peut être acceptée comme meilleur global même si elle est moins bonne, avec une probabilité dépendant d'une température décroissante et d'un autre paramètre $0 < \beta < 1$. Plus la température initiale est grande, plus on aura de chances d'accepter une solution moins bonne. Il en est de même pour β , où un β grand va induire un refroidissement plus lent et donc une probabilité plus grande de sortir de notre optimum local. Le problème principal de l'algorithme PSO étant le blocage dans ces minimum locaux, nous prenons des valeurs élevées de ces hyper-paramètres :

$$\boxed{\beta = 0.99 \quad T_0 = 100}.$$

On propose ainsi d'implémenter ceci à chaque itération k de la boucle while dans notre algorithme :

$k \leftarrow$ l'itération en question
 Calculer la température :

$$T_k \leftarrow T_0 \beta^k$$

Pour une solution candidate s :

$$p \leftarrow \min \left(1, \exp \left(-\frac{f(s) - f(g^k)}{T_k} \right) \right)$$

Tirer un nombre aléatoire r dans $[0, 1]$ de manière uniforme
 Si $r < p$ alors :
 adopter s comme nouveau meilleur chemin global
 FinSi

En implémentant cette solution, on trouve alors les performances suivantes :

Scenario 3 : La longueur du chemin calculé est 1536.0338430221568 Temps CPU effectif : 2789.241375 ms

Scenario 4 : La longueur du chemin calculé est 21491.70485267472 Temps CPU effectif : 3055.165319 ms

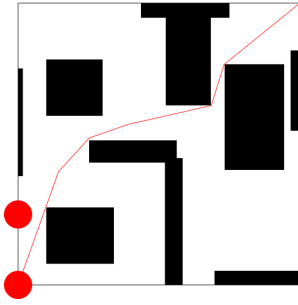


FIGURE 8 – Chemin de longueur 1536 calculé par l'algorithme PSO pour le Scenario 3, le temps de calcul était de 2789.2 ms.

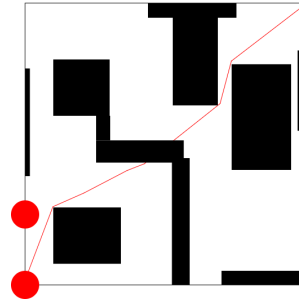


FIGURE 9 – Chemin de longueur 21491.7 calculé par l'algorithme PSO pour le Scenario 4, le temps de calcul était de 3055 ms.

Dans ce cas-ci, il est compliqué d'évaluer pleinement notre solution qui dépend beaucoup de probabilités. En effet, on voit que dans le scénario 3, elle est nettement meilleure que la solution initiale et que dans le scénario 4, la tendance est inverse. Le constat est similaire à l'ajout précédent : l'algorithme est un peu plus lent qu'auparavant. Face à cette amélioration partielle, essayons de rendre l'algorithme plus rapide.

2.4.3 • DIMENSIONAL LEARNING

La modification dimensional learning consiste en cela : si une particule i n'a pas amélioré son meilleur local depuis un certain nombre d'itérations fixé arbitrairement, on tente d'améliorer ce meilleur local en remplaçant une dimension à la fois par la dimension correspondante du meilleur global à la k -ième itération, mais uniquement si la fitness diminue avec cette modification, le but étant d'accélérer la convergence vers la meilleure solution. On propose ainsi d'implémenter le pseudo-code suivant à chaque itération de la boucle while :

Soit max_iter_local l'hyperparamètre qui est déjà implémenté pour les solutions précédentes

Si $iter_last_up[i] > max_iter_local$ alors :

$p \leftarrow$ copie du meilleur local de la particule i

$meilleurlocal \leftarrow f(p)$

 Pour chaque dimension j faire :

 mémoriser l'ancienne valeur $p[j]$

 remplacer $p[j]$ par la valeur du meilleur global $g[j]$

 Si $f(p) < meilleurlocal$ alors :

 accepter la modification

 mettre à jour le meilleur local de la particule

$meilleurlocal \leftarrow f(p)$

$iter_last_up[i] \leftarrow 0$

Si non :
 restaurer l'ancienne valeur $p[j]$
Fin Si
Fin Pour
Fin Si

Avec cette amélioration finale, on trouve donc les performances suivantes :

Scenario 3 :

- La longueur du chemin calculé est 1526.2.
- Temps CPU effectif : 15044 ms.

Scenario 4 :

- La longueur du chemin calculé est 21479.2.
- Temps CPU effectif : 15941 ms

Cette dernière amélioration ne semble vraiment pas convaincante étant donné que l'heuristique améliore la convergence alors que notre principal problème était une convergence trop forte vers un minimum local. Cela permet néanmoins de réduire le nombre d'itérations de la boucle *while* à 2000 pour le scenario 3 tout en gardant des résultats satisfaisant et stable. Cependant, cette heuristique est assez coûteuse en temps de calcul et n'est donc pas très utile même pour des environnements "simples".

2.4.4 • NOS PROPRES HEURISTIQUES

Nous avons essayé d'améliorer l'initialisation de notre algorithme. En effet, si le point intermédiaire 1 se retrouve près de l'arrivée et le point final près du départ, il faudra démêler ce chemin ce qui prendra beaucoup de temps. On répartit donc les points intermédiaires sur la droite entre le départ et l'arrivée et on tire nos vrais points aléatoirement dans une zone plus restreinte. Ainsi, l'ordre des points intermédiaires sera vraisemblablement préservé.

Scenario 3 :

- La longueur du chemin calculé est 1567,2.
- Temps CPU effectif : 224.796508 ms.

En conclusion, les résultats de PSO, même améliorés par les différentes modifications sont assez mitigés. Si l'algorithme est assez rapide pour trouver un résultat correct, et est particulièrement efficace lorsqu'il n'y a pas de corridors trop étroit, il renvoie souvent un chemin optimal qui passe à travers un obstacle. Il n'est donc pas réellement optimal pour résoudre le problème que nous nous étions donnés de base : le problème de path-planing.

3

L'ALGORITHME "RAPIDLY-EXPLORING RANDOM TREE".

3.1 PRÉSENTATION DU PRINCIPE GÉNÉRAL.

Nous allons maintenant essayer d'implémenter un autre algorithme, l'algorithme RRT qui sera potentiellement plus performant en explorant une approche totalement différente. En effet, l'algorithme construit progressivement un arbre de recherche dans l'espace libre, c'est-à-dire en évitant les collisions. À chaque itération, on va tirer un point aléatoire v_r , puis chercher dans l'arbre le nœud le plus proche v_n . Par la suite, on va ajouter un nouveau nœud v en se dirigeant de v_n vers v_r sur une distance maximale δ_s , qui sera l'un de nos hyperparamètres. Si le segment entre v_n et v est sans collision, v est inséré dans l'arbre et a pour parent v_n . Ensuite, l'algorithme tente d'améliorer le coût du chemin via un choix de parent optimal autour de v . On cherche dans un cercle centré sur v de rayon δ_r , qui représente le deuxième hyperparamètre du problème. Si l'on trouve un parent de distance plus proche avec le point initial, on l'assigne comme nouveau parent de v , puis une phase de *rewiring* où certains voisins peuvent changer de parent si passer par v réduit la longueur du chemin total.

3.2 IMPLÉMENTATION POUR NOTRE PROBLÈME.

Nous avons besoin d'une structure de donnée qui puisse efficacement réaliser les trois opérations suivantes : ajouter un point, trouver le point le plus proche, trouver tous les points à une distance fixé. On s'oriente ainsi vers un arbre de recherche, où chaque nœud v est caractérisé par la longueur de son chemin à l'origine, ses coordonnées dans le domaine étudié (x,y) ainsi que ses fils, qui sont stockés dans une liste chaînée du type nœud décrit précédemment. Enfin, le nœud est aussi caractérisé par un type parent, qui correspond simplement à son nœud parent dans l'arbre que nous construisons. Attention il faut bien comprendre que les fils et parent stockés dans chaque nœud ne sont pas ceux de l'arbre de recherche mais ce du chemin vers l'origine. Le parent du nœud est le point qui est juste avant dans le chemin le plus court depuis l'origine et les fils sont tous les points dont le chemin depuis l'origine passe par le point courant. Cette structure nous semble optimale car permettra directement de trouver la longueur de chaque chemin vers un nœud v et donc de résoudre plus facilement le problème de minimisation. Par ailleurs, en gardant les fils nous pouvons facilement propager le changement de distance à l'origine du point courant lorsque qu'un nouveau nœud avantageux est ajouté.

Pour remonter d'un sommet vers l'origine, nous proposons la méthode suivante : en partant du nœud n , on itère sur la caractéristique parent du nœud et on stocke tout dans une liste chaînée contenant les coordonnées de chaque nœud parent par lequel on passe. L'itération s'arrête lorsque le nœud sur lequel on se trouve ne possède pas de parent, on est alors revenu au point d'origine v_s . Voici le pseudo-code de ce que l'on vient de décrire :

Entrée :

— un nœud n appartenant à l'arbre d'origine

Sortie :

— un chemin contenant les coordonnées des points par lesquels on passe depuis n jusqu'au nœud racine

Algorithme :

Si n est nul alors :

retourner un chemin vide

FinSi

Créer un chemin vide *path*

current $\leftarrow n$

Tant que *current* $\neq$ null *faire* :

ajouter les coordonnées du nœud courant au chemin

current $\leftarrow$ *parent*(*current*)

FinTantQue

Renvoyer *path*

Il reste maintenant à voir comment insérer, trouver les points à une distance fixée d'un point et trouver le point le plus proche. Pour cela, notre arbre de recherche va partitionner l'espace, horizontalement pour les niveaux de profondeur pair et verticalement pour ceux impairs. Cela est plus clair avec le pseudo-code de la fonction qui insère un nouveau point à notre arbre de recherche.

Entrées :

- un nœud *n*, ce qui est ici surtout essentiel est les coordonnées (x, y) de ce nœud
- un arbre de recherche *t*
- un entier naturel *depth* représentant la profondeur, en pratique on appellera l'algorithme avec une profondeur 0

Sortie :

- Ne renvoie rien mais insère le nœud *n* dans l'arbre *t*

Algorithme

On vérifie que le problème n'est pas trivial ($n \neq$ null par exemple)

Si *depth* est pair *alors*

Si $x_n < x_{racine}$ *alors*

On appelle la fonction **insert**(*n*, *t*.*left*, *depth* + 1)

sinon on appelle la fonction avec le sous arbre droit

sinon

Si $y_n < y_{racine}$ *alors*

On appelle la fonction **insert**(*n*, *t*.*left*, *depth* + 1)

sinon on appelle la fonction avec le sous arbre droit

Présentons maintenant l'algorithme qui trouve le nœud de l'arbre de recherche le plus proche d'un nouveau point.

Entrées :

- un Point *p*, dont on utilise les coordonnées (x, y)
- un entier naturel *depth* représentant la profondeur
- un arbre de recherche *current* représentant l'arbre courant exploré
- un nœud *best*, représentant le meilleur candidat trouvé jusqu'à présent

Sortie :

- Renvoie le nœud le plus proche de *p* dans le sous-arbre (mise à jour de *best*)

Algorithme

```

Si  $current == \text{null}$  alors
  On retourne  $best$ 
On calcule  $current\_dist$  la distance au carré entre  $p$  et le point du nœud courant
On calcule  $best\_dist$  la distance au carré entre  $p$  et le point de  $best$ 
Si  $current\_dist < best\_dist$  alors
   $best\_dist \leftarrow current\_dist$ 
   $best \leftarrow$  le nœud de  $current$ 
Si  $depth$  est pair alors
   $diff \leftarrow x_p - x_{current}$ 
sinon
   $diff \leftarrow y_p - y_{current}$ 
Si  $diff < 0$  alors
   $nearChild \leftarrow current.left$ 
   $farChild \leftarrow current.right$ 
sinon
   $nearChild \leftarrow current.right$ 
   $farChild \leftarrow current.left$ 
 $best \leftarrow \text{findNearest\_rec}(p, depth + 1, nearChild, best)$  on cherche parmi les nœuds du même côté que  $p$ 
Si  $diff \times diff < \text{distance au carré entre } p \text{ et } best$  alors analyse la distance de  $p$  à la droite de séparation
   $best \leftarrow \text{findNearest\_rec}(p, depth + 1, farChild, best)$  le plus proche voisin peut être dans l'arbre éloigné
On retourne  $best$ 

```

On utilise un algorithme absolument similaire pour trouver les nœuds à une distance fixée d'un point. On a ainsi tous nos algorithmes pour implémenter l'algorithme RRT dont voici le pseudo-code.

Entrées :

- un environnement env (obstacles) et un test de collision $\text{CollisionLibre}(a, b)$ qui renvoie un booléen donnant ou non la présence d'un obstacle entre deux points a et b , on peut considérer une adaptation de la méthode utilisée en partie précédente (avec ici une complexité en le nombre d'obstacles m)
- un point de départ v_s et un point d'arrivée v_g
- un nombre maximal d'itérations max_iter noté q
- un pas d'extension δ_s et un rayon de voisinage δ_r
- une fonction de coût $cout(v)$ donnant le coût du chemin de v_s à v dans l'arbre

Sortie :

- un chemin de v_s à v_g (ou vers le nœud le plus proche de v_g si aucun chemin valable ne mène au point d'arrivée) obtenu en remontant les parents depuis v_g

Algorithme :

```

Créer un arbre  $T$  contenant uniquement le nœud  $v_s$ 
Pour  $k$  allant de 1 à  $max\_iter$ , faire :
   $v_r \leftarrow$  point aléatoire dans le domaine étudié
   $v_n \leftarrow$  nœud de  $T$  le plus proche de  $v_r$ 
   $v \leftarrow$  nouveau nœud obtenu en allant de  $v_n$  vers  $v_r$  sur une distance au plus  $\delta_s$ 
  Si  $\text{CollisionLibre}(v_n, v)$  alors :
     $\text{parent}(v) \leftarrow v_n$ 
    Ajouter  $v$  à l'arbre  $T$ 

```

```

Voisins  $\leftarrow \{u \in T \mid \text{dist}(u, v) \leq \delta_r\}$ 
Pour chaque  $u \in \text{Voisins}$ , faire :
    Si CollisionLibre( $u, v$ ) et  $\text{cout}(u) + \text{dist}(u, v) < \text{cout}(v)$  alors :
        parent( $v$ )  $\leftarrow u$ 
        Mettre à jour  $\text{cout}(v) \leftarrow \text{cout}(u) + \text{dist}(u, v)$ 
    FinSi
FinPour
Pour chaque  $u \in \text{Voisins}$ , faire :
    Si CollisionLibre( $v, u$ ) et  $\text{cout}(v) + \text{dist}(v, u) < \text{cout}(u)$  alors :
        parent( $u$ )  $\leftarrow v$ 
        Mettre à jour  $\text{cout}(u) \leftarrow \text{cout}(v) + \text{dist}(v, u)$ 
    FinSi
FinPour
FinSi
Renvoyer le chemin obtenu en remontant les parents depuis  $v_g$  (ou depuis le nœud de  $T$  le plus proche de  $v_g$ )

```

Intéressons-nous maintenant à la complexité de chaque itération. Notons k l'itération sur laquelle nous travaillons (soit k éléments dans l'arbre dans le pire des cas). À chaque itération, on fait un test de collision donc on a déjà une complexité en $\mathcal{O}(m)$. Par la suite, l'ajout se fait lui en $\mathcal{O}(k)$ dans le pire des cas. Le nombre de voisins est aussi k dans le pire des cas et pour chaque passage dans la boucle (où l'on regarde les voisins du sommet v) on a une complexité en $\mathcal{O}(m)$ car il y a un test de collisions. Par ailleurs, trouver les voisins dans un rayon δ_r et le plus proche voisin sont des opérations de coût linéaire en k . On a donc une complexité par itération en $\mathcal{O}(km)$.

3.3 ANALYSE DE L'ALGORITHME.

3.3.1 • RÉSULTATS.

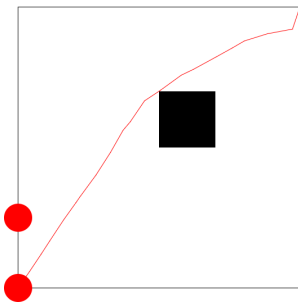


FIGURE 10 – Chemin de longueur 1477 calculé par l'algorithme RRT pour le Scenario 0, le temps de calcul était de 155 ms.

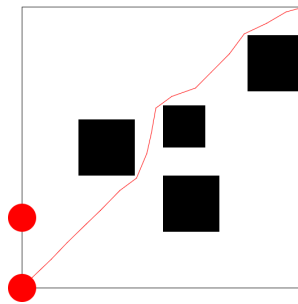


FIGURE 11 – Chemin de longueur 1476 calculé par l'algorithme RRT pour le Scenario 1, le temps de calcul était de 153 ms.

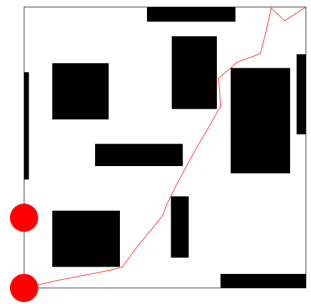


FIGURE 12 – Chemin de longueur 1627.8 calculé par l'algorithme RRT pour le Scenario 2, le temps de calcul était de 218 ms.

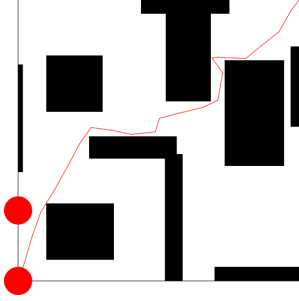


FIGURE 13 – Chemin de longueur 1677 calculé par l'algorithme RRT pour le Scenario 3, le temps de calcul était de 200 ms.

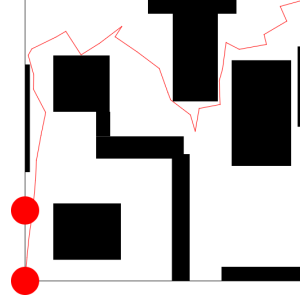


FIGURE 14 – Chemin de longueur 2579.4 calculé par l'algorithme RRT pour le Scenario 4, le temps de calcul était de 4559 ms.

3.4 AMÉLIORATIONS ET HEURISTIQUES.

3.4.1 • PATH OPTIMIZATION

Nous allons implémenter une optimisation naturelle, nous utilisons l'inégalité triangulaire pour raccourcir le chemin. si un voisin n permet d'atteindre v par un chemin plus court que le chemin actuel, alors on remplace le parent de v par n . Cette amélioration repose sur l'inégalité triangulaire : passer directement par un voisin proche peut raccourcir le coût total, à condition que le segment soit sans collision.

Voici donc le pseudo-code qui implémente cette optimisation :

Entrée :

- un nœud nouvellement ajouté v
- un ensemble de voisins $Voisins(v)$ (dans un rayon δ_r)

Sortie :

- le nœud v avec un parent éventuellement amélioré (chemin plus court)

Algorithme :

```

Pour chaque  $n \in Voisins(v)$  faire :
  Si  $dist(n) + \|v - n\| < dist(v)$  et  $CollisionLibre(v, n)$  alors :
     $parent(v) \leftarrow n$ 
     $dist(v) \leftarrow dist(n) + \|v - n\|$ 
  FinSi
FinPour

```

3.4.2 • ANALYSE DES PERFORMANCES ET COMPARAISON AVEC LE RRT BASIQUE.

Intéressons-nous d'abord à la complexité pour chaque itération de la boucle donnée. On teste collision libre (mais c'était déjà fait pour avoir la distance précédemment) et fait la comparaison de distance. Donc à l'itération k on a dans le pire des cas k voisins pour le sommet v , et le test de collision libre est de complexité $\mathcal{O}(m)$, m le nombre de collisions. Donc, la complexité de la boucle décrite ici est en $\mathcal{O}(km)$

On voit donc que notre algorithme RTT comme tel a des performances similaires au PSO basique. Néanmoins, la grande différence réside dans le fait que l'on arrive bien plus souvent à avoir un chemin valable, ne passant pas par des obstacles que avec le PSO. Cet algorithme est ainsi bien plus performant que le précédent.

3.4.3 • INTELLIGENT SAMPLING

Testons une nouvelle approche pour optimiser notre algorithme, celle de choisir astucieusement le point v_r que l'on prenait aléatoirement dans l'algorithme initial. On va procéder ainsi : au lieu d'échantillonner uniformément dans tout l'espace, l'algorithme peut introduire un biais afin d'accélérer la convergence vers l'objectif. Avec une certaine probabilité (ici on choisit 0.10), le point cible v_g est sélectionné directement. Sinon, avec probabilité non-nulle (ici 0.40) on va chercher à se rapprocher d'un obstacle tiré aléatoirement, ce qui est le cœur de notre nouvelle approche, avoir des points plus proches des différents obstacles pour réduire la longueur du chemin total. Sinon, un point est tiré aléatoirement dans l'espace libre. Ce mécanisme permet de guider l'expansion de l'arbre vers une zone pertinente tout en conservant une capacité d'exploration globale, pour ne pas se retrouver bloquer dans un optimum local, problématique déjà présente dans l'étude de l'algorithme PSO faite précédemment.

Voici donc le pseudo-code de notre fonction permettant de choisir v_r :

Entrée :

- un environnement env
- un point d'arrivée (x_e, y_e)

Sortie :

- un point échantillon v_r

Algorithme :

```

Tirer rand uniformément dans  $[0, 1]$ 
Si rand < 0.10 alors :
    Renvoyer le point objectif  $(x_e, y_e)$ 
FinSi
Si rand < 0.40 alors :
    obs ← obstacle tiré aléatoirement dans env
    corner ← un coin aléatoire de obs
    Tirer  $\eta_x$  selon une loi normale centrée réduite
    Tirer  $\eta_y$  selon une loi normale centrée réduite
     $nx \leftarrow corner.x + 15 \cdot \eta_x$ 
     $ny \leftarrow corner.y + 15 \cdot \eta_y$ 
     $p \leftarrow (nx, ny)$ 
    Si  $p$  est dans l'espace libre (env.isPointFree( $p$ )) alors :
        Renvoyer  $p$ 
    FinSi
FinSi
Renvoyer un point aléatoire uniforme dans l'espace (init_rand_point(env))

```


3.4.4 • COMPARAISON AVEC PSO ET CONCLUSION SUR L'ALGORITHME

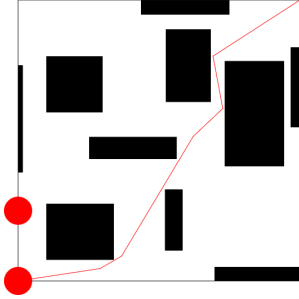


FIGURE 15 – Scenario 2 : le chemin a une longueur initiale de 1585.4 et après optimisation de 1580, temps de calcul : 397.041543 ms.

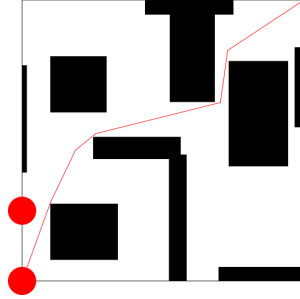


FIGURE 16 – Scenario 3 : le chemin a une longueur initiale de 1702 et après optimisation de 1563, temps de calcul : 397.041543 ms.

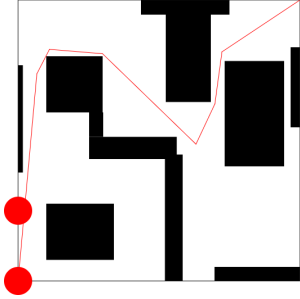


FIGURE 17 – Scenario 4 : le chemin a une longueur initiale de 2258 et après optimisation de 2169, temps de calcul : 397.041543 ms.

On voit donc avec les comparaisons de taille de chemin que nous avons effectué plus haut, que les améliorations proposés sont relativement efficace, la longueur du chemin baisse à chaque fois (même si cela ne dépasse pas un dixième de la taille du chemin total, cela reste non-négligeable).

A l'aide de tout ce que nous avons fait, on peut donc conclure que pour résoudre le problème de path planning, l'algorithme RRT est nettement plus performant. En effet, s'il a des performances similaires vis-à-vis du Run Time, ainsi que de l'ordre de grandeur de la longueur des chemins trouvés, cet algorithme renvoie toujours un chemin fonctionnel, ne traversant pas d'obstacles. Alors qu'avec l'algorithme PSO, on retrouvait une solution fonctionnelle parfois au bout de 10 essais (ce qui en étant pris en compte dans le temps de run total signifie qu'il est d'un ordre de grandeur plus lent que RRT), ce n'est jamais le cas pour RRT qui est nettement plus fiable.

On conclut donc que l'algorithme RRT implémenté en deuxième est plus performant que le premier, l'algorithme PSO.

4

VERS UNE ADAPTATION AUX SITUATIONS RÉELLES : LA GESTION DE PLUSIEURS ROBOTS.

Pour ce problème, nous considérons qu'il y a maintenant deux robots, qui ont rayon R . L'objectif est de trouver un chemin pour chaque robot vers son objectif, tout en garantissant que les deux robots ne vont pas se rentrer dedans. Nous souhaitons utiliser l'algorithme RRT car celui-ci est plus efficace et surtout fournit des chemins valides.

Pour traiter ce problème, nous implémentons l'algorithme RRT dans un espace à 4 dimensions. Nous n'utilisons plus ici un arbre de recherche qui serait bien moins efficace en 4 dimensions mais un simple tableau. Les algorithmes de recherche du plus proche voisin et de recherche de tous les nœuds à une distance fixée sont alors linéaires en la taille de notre tableau.

L'algorithme est alors exactement le même à une différence près : ici on avance à chaque étape de δ_s , alors que précédemment si le nœud tiré v_r était à une distance inférieure à δ_s , nous fixions $v_r = v$. Nous augmentons également la valeur du nombre d'itérations à 10000 car l'espace considéré est bien plus grand (pour le scénario 4 on peut augmenter encore plus ce nombre car il y a beaucoup d'obstacles.) Voici les résultats de notre algorithme :

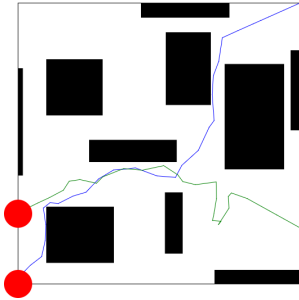


FIGURE 18 – Scénario 2 avec les deux robots activés.

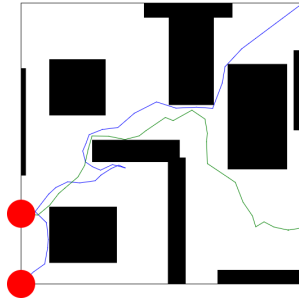


FIGURE 19 – Scénario 3 avec les deux robots activés.

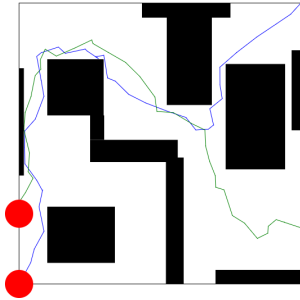


FIGURE 20 – Scénario 4 avec les deux robots activés.

Même pour le scénario 4 avec, le temps de cet algorithme est inférieur à 5 secondes. Ce qui reste bien évidemment grandement optimisable.